

This is a faster mechanism of communication, since after creating and attaching shared memory to a process, you do not need any special function calls to access it. It simply behaves like normal data storage (a variable or array). But if multiple processes simultaneously read or write shared memory, race conditions can occur. Avoid that by using a synchronisation object like a semaphore. (Please refer to any standard OS textbook or the Internet for more details on race conditions and semaphores, if needed.)

Let us now implement the program using shared memory (using System V functions) and a semaphore (using POSIX functions). Algorithms follow:

Sender

```
Generate key (say K1).
Create/Get Access to the Shared Memory (The key K1 is needed here).
Attach the Shared Memory to the process address space.
Create a Semaphore (named put)
Create a Semaphore (named get)
Initialise get to 0.
Initialise put to 1.
while (1) {
    Read from Key board.
    Wait for semaphore put to become 1
    Write User Input to Shared memory.
    Update semaphore get to 1
    If user input was "exit" then break;
}
Delete shared memory.
Delete semaphores get and put.
Program terminates.
```

Receiver

```
Generate key (say K1).
Create/Get Access to the Shared Memory (The key K1 is needed here).
Attach the Shared Memory to the process address space.
Get Access to the Semaphore get.
Get Access to the Semaphore put.
while (1) {
    Wait for semaphore get to become 1
    Read data from Shared Memory
    Update semaphore put to 1
    If read data is "exit" then break the loop.
}
Program terminates.
```

The C code implementations are *shm_sender.c* and *shm_receiver.c*. Compile (please use the linker flag *-lrt* here) and run as before (run the sender first, since the semaphores are created in the sender).

Now let's look at the code. The same key is generated by both programs, so they can refer to the same shared memory segment (just like in the message queue example). Before you can use the shared memory, it needs to be attached to the process address space using the *shmat* function, which returns a pointer that can be used to read or write the shared memory.

To avoid race conditions on simultaneous read/writes to the shared memory, two semaphores are used. Here, the sender creates both the semaphores and initialises them. The function for creating or accessing the POSIX named semaphore is *sem_open*. The semaphore is identified by a name; I have used *get* and *put*. You may use any other name, but use the */* character, so that all processes refer to the same semaphore. Here, if the reader runs first, it will be able to immediately write to the shared memory, since the initial value of the *put* semaphore is 1. After writing data, it updates the *get* semaphore to 1, so that the receiver, which was waiting on the *get* semaphore, can now read data. Similarly, now the sender waits, till the receiver successfully reads the shared memory. The semaphores ensure that no race conditions occur. After the user enters 'exit', the semaphores and the shared memory are deleted using the *sem_unlink* and *semctl* function by the sender.

Observations

Message queues offer more flexibility in terms of message storage and retrieval. Shared memories are faster; after creating and attaching *shm*, you don't need to call special functions to read or write data. However, you need to be cautious about possible race conditions, and use semaphores to prevent them.

I hope you enjoyed reading this introductory article and start using what I've featured in applications. 

References

- [1] Bach, Maurice J: 'The Design of the Unix Operating System', PHI
- [2] Stevens, W Richard: 'Unix Network Programming: Interprocess Communication', Vol-2, 2nd Edition, Pearson Education
- [3] Stevens, W Richard, Rago, Stephen A: 'Advanced Programming in the Unix Environment', 2nd Edition, Pearson Education
- [4] Linux Manual Pages

Acknowledgements

I would like to thank Tanmoy Bandyopadhyay and Vikas Nagpal for their help in reviewing this article.

By: Swati Mukhopadhyay

The author has more than 12 years of experience in academics and corporate training. Her interests are digital logic, computer architecture, computer networking, data structures, Linux and programming languages like C and C++. For any queries or feedback, she can be reached at swati.mukherjee@gmail.com or <http://swatimukhopadhyay.wordpress.com/>.